

Golang Technical Test – Realtime Trading System (Mini Exchange)

Objective

Membangun mini realtime trading backend menggunakan Golang yang mampu:

1. Menyediakan REST API
2. Menyediakan WebSocket untuk data realtime
3. Menangani event streaming (price / trade)
4. Menangani concurrency dengan baik
5. Dirancang untuk asumsi beban produksi skala menengah

Asumsi Beban Sistem

Gunakan asumsi berikut dalam desain:

1. 1000 order per menit
2. 500 WebSocket client aktif
3. Setiap client subscribe 1 sampai 5 stock
4. Broadcast tidak boleh blocking meskipun ada client lambat

Tidak perlu melakukan load test.

Namun desain harus mempertimbangkan asumsi ini dan dijelaskan di README.

Requirements

REST API (HTTP)

Buat beberapa endpoint berikut:

1. Create Order
 - a. Endpoint untuk membuat order (buy/sell)
 - b. Sistem harus aman terhadap concurrent order submission.
 - c. Input minimal:
 - Stock Code
 - Side (BUY / SELL)
 - Price
 - Quantity
2. Get Order List
 - a. Mengambil daftar order yang sudah dibuat
 - b. Bisa filter by:
 - Stock
 - Status (optional)
3. Get Trade History: Menampilkan history trade yang sudah terjadi
4. Get Market Info (Snapshot): Agar client bisa render data awal tanpa menunggu WS event pertama, sediakan endpoint market snapshot:
 - a. Get Ticker/Price (contoh: last price, change, volume)
 - b. Get Order Book (bid/offer depth)
 - c. Get Recent Trades (last N trades) (Opsional)

Catatan:

Market snapshot ini boleh bersumber dari simulasi internal, atau dari public API (lihat section rekomendasi di bawah).

MATCHING ENGINE (CORE LOGIC)

Implementasikan simple matching engine:

1. Order BUY akan match dengan SELL jika harga sesuai
2. Jika match:
 - a. Buat trade
 - b. Update status order
3. Handle:
 - a. Partial fill (optional bonus)
 - b. FIFO matching (optional bonus)

Tambahan wajib:

1. Matching engine harus aman terhadap race condition
2. Jelaskan bagaimana memastikan konsistensi data saat order masuk secara bersamaan

WEBSOCKET (REAL TIME)

Buat WebSocket server dengan kemampuan:

1. Client bisa subscribe ke:
 - a. price/ticke
 - b. trade
 - c. order update (optional bonus)
2. Realtime channels minimal (disarankan)
Agar jelas dan mudah dites, minimal dukung channel berikut:
 - a. market.ticker (harga terakhir per Stock)
 - b. market.trade (stream trade per Stock)
 - c. market.orderbook (opsional bonus) (depth bid/offer)
 - d. order.update (opsional bonus) (status order per user / per client)

Catatan: Saat client connect, client kirim message subscribe/unsubscribe (format bebas tapi konsisten).

3. Server harus:
 - a. Broadcast update secara realtime
 - b. Mengirim data ketika:
 - ada trade terjadi
 - ada perubahan harga
4. Handle multiple client connection
 - a. Tidak boleh blocking
 - b. Harus scalable secara concurrency

REALTIME PRICE SIMULATION

Buat simulasi data:

1. Harga berubah secara periodik / berdasarkan trade
2. Generate event secara realtime
3. Broadcast ke semua subscriber

Jelaskan di README bagaimana simulasi bekerja.

CONCURRENCY & PERFORMANCE

1. Gunakan:
 - a. Goroutine
 - b. Channel
2. System harus:
 - a. tidak race condition
 - b. tidak blocking
 - c. mampu handle multiple request & connection

Tambahan wajib:

1. Jelaskan potensi bottleneck Utama
2. Jelaskan strategi scaling horizontal
3. Jelaskan bagaimana mencegah goroutine leak

DATA STORAGE

1. Boleh menggunakan:
 - a. in-memory (wajib)
 - b. Redis (optional bonus)
 - c. Database (optional bonus)

Technical Constraints

1. Gunakan Golang
2. Clean architecture (separation layer diutamakan) (Nilai tambah)
3. Code harus readable & maintainable
4. Handle error dengan baik

Expected Output

Candidate harus mengumpulkan:

1. Source Code
 - a. Git repository (public/private)
 - b. Struktur project jelas
2. README.md, wajib berisi:
 - a. Cara menjalankan project
 - b. Design arsitektur (singkat)
 - c. Flow system

- d. Assumption yang digunakan
- e. Penjelasan potensi race condition
- f. Strategi broadcast non blocking
- g. Tiga bottleneck utama dan cara mengatasinya
- 3. API Documentation
 - a. List endpoint
 - b. Cara penggunaan
- 4. WebSocket Documentation
 - a. Cara connect
 - b. Cara subscribe
 - c. Format message (bebas, tapi konsisten)
 - d. Rekomendasi tools dan cara testing websocket di tools tersebut

Rekomendasi Open API / WebSocket Public (Gratis) untuk Testing

Bagian ini opsional. Kandidat boleh tetap memakai simulasi data internal. Kalau ingin pakai sumber data eksternal agar harga/trade terasa "real", berikut opsi yang umum dipakai.

1. Finnhub — REST + WebSocket (free tier, butuh API key)
 Cocok untuk: realtime market data lintas stocks / forex / crypto (tergantung batasan free tier).
 Referensi: Finnhub: <https://finnhub.io/>
2. Binance (Crypto) — Public WebSocket tanpa API key
 Cocok untuk: realtime ticker / trades / orderbook.
 Kelebihan: benar-benar public untuk market data, tidak perlu login.
 Referensi:
 - Docs WebSocket Streams: <https://developers.binance.com/docs/binance-spot-api-docs/web-socket-streams>
 - Market Data Only URLs (tanpa auth):
https://developers.binance.com/docs/binance-spot-api-docs/faqs/market_data_only
3. Kraken (Crypto) — Public WebSocket tanpa API key
 Cocok untuk: realtime ticker / order book / trades.
 Public WS (unauthenticated): wss://ws.kraken.com/
 Referensi:
 Kraken WebSocket API FAQ: <https://support.kraken.com/articles/360022326871-kraken-websocket-api-frequently-asked-questions>
4. Alpaca — Paper trading (gratis) + WebSocket untuk order/account updates (butuh API key)
 Cocok untuk: uji flow submit order → status update di environment paper (bukan anonim, tapi gratis daftar).

Referensi:

- Paper Trading: <https://docs.alpaca.markets/docs/paper-trading>
- WebSocket Streaming (trade/account/order updates): <https://docs.alpaca.markets/docs/websocket-streaming>

Catatan penting (agar sesuai scope technical test)

1. *Market data public umumnya hanya untuk stream satu arah. Untuk "transactional trading dua arah" biasanya perlu API key, atau kandidat implement sendiri di mini exchange.*
2. *Kalau kandidat pakai data crypto dari provider di atas, tinggal mapping Stock (mis. BTCUSDT) ke Stock di sistem kandidat.*

Bonus Point (Nice to Have)

1. Menggunakan message broker (Kafka / NATS)
2. Menggunakan Redis untuk pub/sub
3. Implement rate limiting
4. Implement authentication (JWT)
5. Logging & monitoring
6. Unit test

Evaluation Criteria

1. Arsitektur & design
2. Pemahaman concurrency (goroutine/channel)
3. WebSocket handling
4. Code quality & readability
5. Problem solving
6. Realtime handling approach
7. Membuat backend golang mengikuti (state management / clean architecture) sesuai dengan standard internasional

Notes

1. Tidak perlu UI / frontend
2. Fokus pada backend & realtime system
3. Tidak perlu perfect matching engine (logic sederhana cukup)
4. Yang penting: cara berpikir & struktur system